H A C K M A T C H

Find the Right Team. Build Better Projects.

PRODUCT REQUIREMENTS DOCUMENT

Version 1.0 • Confidential

Deployment: Netlify (FE) + Vercel Serverless (BE) + MongoDB Atlas + Cloudinary

Table of Contents

1. Executive Summary

HackMatch is a startup-grade SaaS platform built specifically for college students who participate in hackathons, final-year projects, startup competitions, research projects, and open-source initiatives. The platform addresses the single most painful friction point in student technical collaboration: finding, evaluating, and recruiting reliable teammates with complementary skill sets.

Existing tools — LinkedIn, WhatsApp groups, Discord servers, Telegram communities, and college notice boards — are either too broad or too unstructured. None of them offer intelligent, skill-aware team formation, built-in project management context, or verifiable student credentials.

Platform Concept:
LinkedIn + Tinder + GitHub for Student Team Formation

Key Value Propositions:

- Skill-verified student profiles with GitHub-style project portfolio
- AI-informed teammate recommendation engine scored across 7 dimensions
- Structured team creation, application, and invitation lifecycle
- Real-time Socket.io team chat with read receipts and typing indicators
- Hackathon discovery and team-building within event context
- Full RBAC: Student, Team Leader, Organizer, Admin
- Mobile-first responsive UI with premium SaaS aesthetics

Product Name	HackMatch
Tagline	Find the Right Team. Build Better Projects.
Primary Audience	College / Engineering / BCA / MCA students aged 18–25
Frontend Stack	React 18 + Vite, Tailwind CSS, DaisyUI, Framer Motion
Backend Stack	Node.js, Express.js, MongoDB Atlas, Mongoose, JWT, Socket.io
Deployment	Netlify (FE) • Vercel Serverless Functions (BE) • Cloudinary (Media)
MVP Timeline	16 weeks across 4 phases
Document Version	1.0 – Implementation Ready

2. Product Vision & User Personas

2.1 Product Vision

HackMatch will become the definitive platform for student technical collaboration in India and beyond. Within 24 months of launch the platform targets 100,000 registered students across 500+ colleges, 10,000 active teams, and partnerships with 200+ hackathon organizers. Revenue streams include organizer SaaS subscriptions, premium student profiles, and eventual recruiter access.

2.2 Primary User Personas

Persona A – The Builder (Primary)

Arjun Mehta, 21 | B.Tech CSE, Year 3 | IIT Delhi

- Attends 6–8 hackathons per year, frequently solo
- Strong backend skills (Node.js, Python) but needs a frontend partner and a designer
- Frustrated by posting in WhatsApp groups and getting no quality response
- Needs: fast skill-filtered teammate search, ability to showcase past projects, direct DM to shortlisted candidates

Persona B – The Seeker (Primary)

Priya Nair, 20 | BCA Year 2 | Christ University Bangalore

- Talented UI/UX designer and React developer, first hackathon season
- Wants to join an established team but has no network
- Needs: discover open teams, apply with profile, receive team invitations

Persona C – The Organizer (Secondary)

Ravi Sinha, 28 | Innovation Cell Lead | BITS Pilani

- Organizes 3 major hackathons per year with 500+ participants
- Needs: event management portal, team registration, participant communication
- Pain: manual team formation coordination via email and Google Sheets

Persona D – The Admin

Platform admin managing user safety, content moderation, and analytics

- Needs: full dashboard with CRUD on all entities, reports management, user bans

2.3 User Roles & RBAC

Role	Who	Key Permissions	Restrictions
Student	All registered users	Profile, browse teams, apply, chat, review	Cannot manage events or admin panels
Team Leader	Student who created a team	All Student perms + create team, invite, manage members, post roles	Cannot manage other teams
Organizer	Verified event organizer	Create hackathons, manage events, view participant teams	Cannot manage user accounts globally
Admin	Platform staff	Full CRUD all entities, ban users, resolve reports, view analytics	N/A – highest privilege

3. System Architecture

3.1 High-Level Architecture

HackMatch follows a decoupled three-tier architecture: a React SPA hosted on Netlify, a stateless Express.js API deployed as Vercel Serverless Functions, and a MongoDB Atlas cluster. Real-time features (chat, notifications, presence) are handled by a Socket.io layer that can be run as a separate long-running process on a small VPS (e.g., Railway or Render) separate from the stateless Vercel functions, since Vercel Serverless Functions do not support persistent WebSocket connections natively.

Layer	Technology	Host	Purpose
Client (SPA)	React 18 + Vite	Netlify CDN	UI rendering, routing, state management
REST API	Express.js (ESM)	Vercel Serverless	Business logic, CRUD, auth, file handling
WebSocket	Socket.io server	Render / Railway (Node process)	Real-time chat, notifications, presence
Database	MongoDB Atlas M10+	AWS ap-south-1	Persistent data store, indexing
Media CDN	Cloudinary	Cloudinary global CDN	Image/file upload, transformation, delivery
Email	Nodemailer + Gmail SMTP	Transactional	OTP, welcome, notification emails
Cache	MongoDB connection cache (module-level)	Vercel runtime	Reuse DB connections across invocations

3.2 Backend Folder Structure

backend/

config/

- db.js – MongoDB Atlas connection with module-level caching for Vercel cold starts
- cloudinary.js – Cloudinary SDK initialization
- nodemailer.js – SMTP transporter configuration

controllers/

- authController.js – register, login, logout, verifyOTP, forgotPassword, resetPassword, refreshToken
- userController.js – getProfile, updateProfile, uploadAvatar, deleteAccount, searchUsers
- teamController.js – createTeam, updateTeam, deleteTeam, getTeam, listTeams, addRole, removeRole
- applicationController.js – applyToTeam, withdrawApplication, reviewApplication, listApplications
- invitationController.js – sendInvitation, acceptInvitation, rejectInvitation, cancelInvitation
- messageController.js – getConversation, getGroupMessages, deleteMessage (REST history)
- notificationController.js – getNotifications, markRead, markAllRead, deleteNotification

- projectController.js – createProject, updateProject, deleteProject, getProject, listProjects
- hackathonController.js – createHackathon, updateHackathon, deleteHackathon, listHackathons, registerTeam
- reviewController.js – createReview, updateReview, deleteReview, getUserReviews
- reportController.js – createReport, listReports, resolveReport (admin)
- adminController.js – listUsers, banUser, unbanUser, getAnalytics, manageOrganizers
- matchController.js – getRecommendations, refreshScore

routes/

- auth.routes.js, user.routes.js, team.routes.js, application.routes.js
- invitation.routes.js, message.routes.js, notification.routes.js
- project.routes.js, hackathon.routes.js, review.routes.js, report.routes.js
- admin.routes.js, match.routes.js

models/

- User.js, Team.js, TeamApplication.js, TeamInvitation.js
- Message.js, Notification.js, Project.js, Hackathon.js
- Skill.js, Review.js, Report.js, Admin.js

middleware/

- auth.middleware.js – verifyAccessToken, verifyRefreshToken
- role.middleware.js – requireRole(...roles), requireOwner
- upload.middleware.js – Multer config (memory storage, file type/size validation)
- rateLimit.middleware.js – express-rate-limit presets (strict for auth, moderate for API)
- validate.middleware.js – express-validator chains for each route
- errorHandler.middleware.js – centralised error formatting, Sentry hook

utils/

- generateTokens.js – sign accessToken (15 min) + refreshToken (7 days)
- sendEmail.js – Nodemailer wrappers for OTP, welcome, notification templates
- matchingEngine.js – computeMatchScore(currentUser, candidateUser) → { score, breakdown }
- cloudinaryHelpers.js – uploadBuffer, deleteAsset
- pagination.js – buildPaginationMeta(total, page, limit)
- sanitize.js – mongo-sanitize wrapper, xss-clean wrapper

.env – All secrets (never committed)

index.js – Express app setup, route registration, Socket.io attachment, server listen

initAdmin.js – One-time seed script to create the first Admin account

3.3 Frontend Folder Structure

src/

Components/

- Navbar.jsx – Responsive top navigation, notification bell, user avatar menu
- Footer.jsx – Site footer with links
- HeroSection.jsx – Animated landing hero with floating blobs, gradient text
- FeatureCard.jsx – Animated feature showcase card (Framer Motion card tilt)
- TeamCard.jsx – Animated team listing card with skill tags, open roles badge

- UserCard.jsx – Profile card with match score, skill chips, quick-invite button
- SkillBadge.jsx – Pill badge for a single skill with level indicator
- ProjectCard.jsx – Project showcase card with tech stack icons, GitHub link
- HackathonCard.jsx – Hackathon listing card with deadline countdown
- NotificationDropdown.jsx – Bell dropdown with real-time socket updates
- ChatWindow.jsx – Full chat UI (messages, input, typing indicator, read receipts)
- ChatSidebar.jsx – Conversation list for 1:1 and group chats
- ApplicationModal.jsx – Apply-to-team modal with cover message input
- InviteModal.jsx – Send invitation to a student modal
- ReviewModal.jsx – Rate & review a teammate after project
- ReportModal.jsx – Report a user or content modal
- LoadingSkeleton.jsx – Reusable skeleton cards for lazy loading
- StatsCounter.jsx – Animated counting statistics widget
- AvatarUploader.jsx – Cropper + Cloudinary upload component
- ProtectedRoute.jsx – Route guard checking auth context
- RoleGuard.jsx – Renders children only for specified roles
- Pagination.jsx – Page controls component
- SearchBar.jsx – Debounced global search input
- FilterPanel.jsx – Skill/college/year filter sidebar
- ConfirmDialog.jsx – Reusable confirm/cancel modal
- ErrorBoundary.jsx – React error boundary with fallback UI
- ScrollReveal.jsx – Intersection Observer wrapper for scroll animations

Pages/

- LandingPage.jsx – Public marketing page with hero, features, stats, CTA
- RegisterPage.jsx – Multi-step registration (credentials → profile → skills)
- LoginPage.jsx – Login form with remember-me, forgot-password link
- VerifyOTPPage.jsx – OTP input grid, resend timer
- ForgotPasswordPage.jsx – Email input to trigger reset OTP
- ResetPasswordPage.jsx – OTP + new password form
- DashboardPage.jsx – Post-login home with recommended teammates, active teams, notifications
- ProfilePage.jsx – Own profile view/edit (bio, skills, projects, reviews)
- PublicProfilePage.jsx – Another user's profile (view-only + invite/message buttons)
- EditProfilePage.jsx – Full profile editor with skill manager, avatar upload
- TeamsPage.jsx – Browse/search/filter all public teams
- TeamDetailPage.jsx – Single team page (members, open roles, apply button)
- CreateTeamPage.jsx – Multi-step team creation wizard
- ManageTeamPage.jsx – Leader panel: applications, invitations, member management
- HackathonsPage.jsx – Browse/filter hackathons
- HackathonDetailPage.jsx – Single hackathon page with team registration
- CreateHackathonPage.jsx – Organizer: create hackathon form
- ManageHackathonPage.jsx – Organizer: view registrations, send announcements
- ProjectsPage.jsx – Browse all public student projects
- ProjectDetailPage.jsx – Single project detail with tech stack, contributors
- CreateProjectPage.jsx – Add a new project to portfolio
- DiscoverPage.jsx – AI-powered teammate recommendations with filter controls
- ChatPage.jsx – Full-screen chat layout (sidebar + window)
- NotificationsPage.jsx – Full notifications list with filters

- AdminDashboardPage.jsx – Admin home with summary widgets
- AdminUsersPage.jsx – User table with search, ban, role assignment
- AdminTeamsPage.jsx – Teams management table
- AdminReportsPage.jsx – Moderation queue for user reports
- AdminAnalyticsPage.jsx – Charts: registrations, DAU, team formation rate
- AdminHackathonsPage.jsx – All hackathons management
- OrganizerDashboardPage.jsx – Organizer home with event summary
- SettingsPage.jsx – Account settings: password, notifications, privacy, delete account
- NotFoundPage.jsx – 404 page with navigation CTA

Context/

- AuthContext.jsx – user state, login(), logout(), token refresh logic, loading flag
- SocketContext.jsx – Socket.io client instance, connection lifecycle, event emitters
- NotificationContext.jsx – unread count, notifications array, real-time updates via socket
- ChatContext.jsx – active conversations, messages map, send/receive via socket
- ThemeContext.jsx – light/dark theme toggle, persisted to localStorage

App.jsx – React Router DOM route tree, context providers, global toast setup

main.jsx – ReactDOM.createRoot, StrictMode

4. Database Design

4.1 MongoDB Connection Caching (Vercel)

Vercel Serverless Functions are stateless; each invocation may spin up a new Node.js runtime. A naive approach creates a new MongoClient connection per request, exhausting the Atlas connection limit under load. The recommended pattern uses a module-level global variable to cache the connection promise:

Strategy: In config/db.js, declare `let cached = global._mongoClientPromise`. On first invocation, create a MongoClient, store the promise in the global, and return it. On subsequent invocations within the same runtime, return the cached promise. This typically reduces cold-start connection overhead from ~500ms to <10ms on warm lambdas.

- Atlas cluster tier M10+ supports up to 1,500 connections; configure maxPoolSize: 10 per function instance
- Set serverSelectionTimeoutMS: 5000 and connectTimeoutMS: 10000 for resilience
- Enable Atlas IP Access List to whitelist Vercel's dynamic egress IPs via 0.0.0.0/0 for serverless or use VPC peering for production security

4.2 User Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	MongoDB primary key	Index: default _id
email	String	Yes	Unique login identifier	Unique Index, lowercase
passwordHash	String	Yes	Bcrypt hash (rounds=12)	Never returned in API
role	String Enum	Yes	student|teamLeader| organizer|admin	Index
isEmailVerified	Boolean	Yes	OTP confirmation status	Default: false
emailOTP	String	No	6-digit OTP (hashed)	TTL managed by expiry field
emailOTPExpiry	Date	No	OTP expiry timestamp	
refreshTokens	[String]	No	Hashed refresh token store	Max 5 entries
firstName	String	Yes	Display name part 1	
lastName	String	Yes	Display name part 2	
avatar	String	No	Cloudinary URL	
bio	String	No	Profile bio (max 500 chars)	
college	String	Yes	College/University name	Text Index
degree	String	Yes	B.Tech / BCA / MCA etc.	
branch	String	Yes	CSE / IT / ECE etc.	
year	Number	Yes	1–5	Index
skills	[ObjectId]	No	Ref: Skill (with level)	Index
interests	[String]	No	Tags: Web, AI, Mobile etc.	

Field	Type	Required	Description	Constraints / Index
githubURL	String	No	GitHub profile link	
linkedinURL	String	No	LinkedIn profile link	
portfolioURL	String	No	Portfolio website	
hackathonsAttended	Number	No	Self-reported count	Default 0
availability	String Enum	No	available|busy|not_looking	Index, Default: available
matchScore	Number	No	Cached composite score	Updated by engine
isActive	Boolean	Yes	Soft delete / ban flag	Default: true, Index
lastSeen	Date	No	Last socket connection	Index
createdAt	Date	Auto	Mongoose timestamps	Index
updatedAt	Date	Auto	Mongoose timestamps	

4.3 Team Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
name	String	Yes	Team display name	Text Index, max 80 chars
slug	String	Auto	URL-safe unique identifier	Unique Index
description	String	Yes	Team/project description	Max 1000 chars
leader	ObjectId	Yes	Ref: User (team creator)	Index
members	[{user, role, joinedAt}]	No	Array of member objects	Embedded
openRoles	[String]	No	Roles being recruited for	e.g., ['Frontend Dev','ML Engineer']
skills	[String]	No	Required tech skills	Index (multi-key)
projectType	String Enum	Yes	hackathon|fyp|startup|research|opensource	Index
hackathon	ObjectId	No	Ref: Hackathon (if applicable)	Index
maxSize	Number	Yes	Maximum team size	Default: 5
isRecruiting	Boolean	Yes	Currently accepting applications	Index, Default: true
isPublic	Boolean	Yes	Visible in discovery	Default: true
status	String Enum	Yes	forming|active|completed|disbanded	Index
college	String	No	College affiliation	Index
tags	[String]	No	Free-form tags	
avatarURL	String	No	Team logo/avatar	

Field	Type	Required	Description	Constraints / Index
createdAt	Date	Auto	Timestamp	Index
updatedAt	Date	Auto	Timestamp	

4.4 TeamApplication Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
team	ObjectId	Yes	Ref: Team	Index
applicant	ObjectId	Yes	Ref: User	Index
role	String	Yes	Role applied for	
coverMessage	String	No	Application message	Max 500 chars
status	String Enum	Yes	pending|accepted|rejected|withdrawn	Index
reviewedBy	ObjectId	No	Ref: User (leader)	
reviewedAt	Date	No	Decision timestamp	
createdAt	Date	Auto		Index

4.5 TeamInvitation Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
team	ObjectId	Yes	Ref: Team	Index
invitedBy	ObjectId	Yes	Ref: User (leader)	
invitee	ObjectId	Yes	Ref: User	Index
role	String	No	Suggested role	
message	String	No	Personal note	Max 300 chars
status	String Enum	Yes	pending|accepted|rejected|expired	Index
expiresAt	Date	Yes	Invitation TTL	Default: +7 days
createdAt	Date	Auto		Index

4.6 Message Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
conversationId	String	Yes	Compound key: sorted userIds or teamId	Index (critical)
sender	ObjectId	Yes	Ref: User	Index

Field	Type	Required	Description	Constraints / Index
type	String Enum	Yes	text|image|file|system	
content	String	No	Text content (max 2000)	
mediaURL	String	No	Cloudinary URL for media	
readBy	[{user, readAt}]	No	Read receipt tracking	
isDeleted	Boolean	Yes	Soft delete	Default: false
createdAt	Date	Auto		Index – sorted desc for pagination

4.7 Notification Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
recipient	ObjectId	Yes	Ref: User	Index (compound with createdAt)
type	String Enum	Yes	application|invitation| message|review|system	Index
title	String	Yes	Short notification title	
body	String	Yes	Notification body text	
link	String	No	Deep link path (e.g. /teams/abc)	
isRead	Boolean	Yes	Read status	Default: false, Index
metadata	Mixed	No	Extra context (teamId, userId, etc.)	
createdAt	Date	Auto		Index desc, TTL: 90 days

4.8 Project Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
owner	ObjectId	Yes	Ref: User	Index
title	String	Yes	Project title	Text Index
description	String	Yes	Project summary	Max 2000 chars
techStack	[String]	Yes	Technologies used	Index
githubURL	String	No	Repository link	
demoURL	String	No	Live demo link	
thumbnailURL	String	No	Cloudinary thumbnail	
collaborators	[ObjectId]	No	Ref: User[]	

Field	Type	Required	Description	Constraints / Index
hackathon	ObjectId	No	Ref: Hackathon	Index
tags	[String]	No	Free-form tags	
isPublic	Boolean	Yes	Visible to all users	Default: true
status	String Enum	Yes	in_progress|completed| archived	
createdAt	Date	Auto		Index

4.9 Hackathon Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
organizer	ObjectId	Yes	Ref: User (organizer)	Index
title	String	Yes	Event name	Text Index
description	String	Yes	Full description	
mode	String Enum	Yes	online|offline|hybrid	
startDate	Date	Yes	Event start	Index
endDate	Date	Yes	Event end	
registrationDeadline	Date	Yes	Apply by date	Index
venue	String	No	Location (offline)	
websiteURL	String	No	Official website	
bannerURL	String	No	Cloudinary banner	
teamSizeMin	Number	Yes	Min team size	Default: 1
teamSizeMax	Number	Yes	Max team size	Default: 5
prizes	[{rank, prize}]	No	Prize tiers	
tags	[String]	No	Theme tags	
registeredTeams	[ObjectId]	No	Ref: Team[]	
isActive	Boolean	Yes	Visible to public	Default: true
createdAt	Date	Auto		Index

4.10 Skill Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
name	String	Yes	Skill name (React, Python…)	Unique Index, lowercase
category	String	Yes	frontend|backend|mobile|	Index

Field	Type	Required	Description	Constraints / Index
			ml|design|devops|other	
icon	String	No	Icon identifier	
usageCount	Number	Auto	Popularity counter	Index desc

4.11 Review Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
reviewer	ObjectId	Yes	Ref: User (author)	Index
reviewee	ObjectId	Yes	Ref: User (subject)	Index
team	ObjectId	Yes	Ref: Team (context)	
rating	Number	Yes	1–5 stars	
comment	String	No	Written review	Max 500 chars
tags	[String]	No	Good communicator, Reliable etc.	
isVisible	Boolean	Yes	Hidden if reported	Default: true
createdAt	Date	Auto		Index

4.12 Report Schema

Field	Type	Required	Description	Constraints / Index
_id	ObjectId	Auto	Primary key	
reporter	ObjectId	Yes	Ref: User	
targetType	String Enum	Yes	user|team|review|message	
targetId	ObjectId	Yes	ID of reported entity	Index
reason	String Enum	Yes	spam|harassment|fake| inappropriate|other	
description	String	No	Additional details	Max 500
status	String Enum	Yes	pending|reviewed|resolved| dismissed	Index
resolvedBy	ObjectId	No	Ref: Admin User	
resolution	String	No	Admin action taken	
createdAt	Date	Auto		Index

5. Feature Specifications

5.1 Authentication Module

Registration Flow

1. User lands on /register and completes Step 1: email, password, confirm password
2. Step 2: firstName, lastName, college, degree, branch, year
3. Step 3: skills selection (min 1, max 20), interests, optional social links
4. On submit, POST /api/auth/register creates user (isEmailVerified: false), sends OTP email via Nodemailer
5. User redirected to /verify-otp; enters 6-digit code within 10-minute window
6. POST /api/auth/verify-otp: validates OTP, sets isEmailVerified: true, issues accessToken (httpOnly cookie, 15 min) + refreshToken (httpOnly cookie, 7 days)
7. User directed to DashboardPage

Login / Logout / Token Refresh

- POST /api/auth/login: validates credentials, checks isEmailVerified, checks isActive, issues tokens
- POST /api/auth/logout: clears httpOnly cookies, removes refreshToken from DB array
- POST /api/auth/refresh: reads refreshToken cookie, validates against hashed store, issues new accessToken
- Middleware verifyAccessToken reads Authorization: Bearer <token> header OR cookie; role middleware follows
- Frontend: Axios interceptor catches 401, calls /refresh transparently, retries original request

Password Reset OTP Flow

8. POST /api/auth/forgot-password: validate email exists, generate 6-digit OTP, hash + store with 10-min expiry, send email
9. POST /api/auth/verify-reset-otp: validate OTP, return short-lived reset token (5 min JWT)
10. POST /api/auth/reset-password: verify reset token, hash new password, invalidate all refresh tokens

Security Considerations

- OTPs are bcrypt-hashed before storage; timing-safe comparison on verify
- Max 5 OTP attempts per 15-minute window (rate limiter)
- refreshTokens array limited to 5; oldest evicted on overflow (device logout)
- All auth routes behind 10 req/min strict rate limit

5.2 User Profile Module

Profile Components

- Header: avatar, name, college, year, availability badge, match score (if viewing from Discover)
- Skills Section: categorised skill chips with proficiency level (beginner/intermediate/advanced)

- Projects Portfolio: grid of ProjectCards linked to owner's projects
- Reviews Section: star rating average, individual review cards, pagination
- Hackathons Attended: badge count + list of hackathons participated in
- Social Links: GitHub, LinkedIn, Portfolio, with icon buttons

Edit Profile

- Avatar upload: Multer processes file in memory → Cloudinary upload → store URL; old asset deleted
- Skills manager: searchable autocomplete from Skill collection, drag to reorder, set proficiency level
- Inline edit for bio, social links, college info, availability status
- Change password requires current password confirmation

5.3 Team Creation Module

User Flow

11. Team Leader navigates to /teams/create
12. Step 1: Team name, description, project type (hackathon / FYP / startup / research / opensource)
13. Step 2: Attach to hackathon (optional), set max team size (2–10), set required skills
14. Step 3: Add open roles (e.g., 'Backend Developer – Node.js', 'ML Engineer – PyTorch')
15. Step 4: Upload team avatar (optional), set visibility (public/private), confirm creation
16. POST /api/teams creates team, leader added as first member with role: 'Leader'
17. Team slug auto-generated from name; collision appends random suffix

Edge Cases

- A user can lead at most 3 simultaneous active teams
- Team name must be unique within same hackathon context
- Deleting a team with accepted members triggers notification to all members

5.4 Team Recruitment Module

Open Roles Management

- Leader adds/edits/removes open roles from ManageTeamPage
- Each role has: title, required skills, description, slots available (1–5)
- isRecruiting flag can be toggled; when false, application button hidden on TeamDetailPage

Application Review

- Applications list in ManageTeamPage with filter: pending / accepted / rejected
- Each application card shows: applicant profile snapshot, match score, cover message, action buttons
- Accepting an application: adds user to team members, sends acceptance notification, updates role slot count

- Rejecting: sends rejection notification with optional custom message

5.5 Team Discovery Module

Discovery Filters

- Project type (hackathon, FYP, startup, research, opensource)
- Required skills (multi-select from Skill collection)
- Hackathon (filter teams recruiting for specific event)
- College / Location
- Team size (current members vs max)
- Recruiting status (show only open teams)

Sorting

- Newest first (default)
- Best match (requires auth; uses Matching Engine score against team skills)
- Most active (last activity timestamp)

5.6 Skill Matching Engine

The matching engine computes a compatibility score between the current authenticated user and a candidate teammate (or team). See Section 6 for full algorithm detail.

5.7 Team Application System

Student Apply Flow

18. Student opens TeamDetailPage; sees open roles and 'Apply' button (hidden if already applied or member)
19. ApplicationModal opens: student selects target role, writes cover message (optional, max 500 chars)
20. POST /api/applications: checks student not already member, not duplicate pending application, team isRecruiting
21. Leader receives real-time notification via socket + email notification
22. Student can withdraw pending application (PATCH /api/applications/:id/withdraw)

Edge Cases

- Max 10 pending applications per student at any time
- Application auto-expires if team disbands or hackathon registration closes
- Student banned from applying to a team if previously rejected twice by same team

5.8 Team Invitation System

Leader Invite Flow

23. Leader browses DiscoverPage or PublicProfilePage, clicks 'Invite to Team'
24. InviteModal: select which team (if leader of multiple), select role, add personal message
25. POST /api/invitations: validates invitee not already member, not duplicate pending invitation
26. Invitee receives real-time notification + email
27. Invitee can accept (→ joins team) or reject; invitations expire after 7 days

5.9 Real-Time Chat System

Detailed architecture in Section 7. Feature requirements:

- One-to-One DMs: accessible from any user's profile or existing conversation
- Team Group Chat: created automatically when team is formed; all members have access
- Message types: text, image (Cloudinary), file attachment
- Typing indicator: debounced 'typing...' event with 3-second timeout
- Read receipts: read by list per message, shown as avatar stacks
- Online presence: green dot on avatars for users connected via socket
- Message deletion: sender can delete within 60 minutes; replaced with 'Message deleted' placeholder
- Infinite scroll pagination: load 20 messages per batch, scroll-to-top triggers next batch

5.10 Project Showcase Module

Project CRUD

- Student creates projects linked to their profile; optional hackathon association
- Fields: title, description, tech stack (chips), GitHub URL, demo URL, thumbnail (Cloudinary), collaborators (tag teammates), status
- Public projects indexed for full-text search
- ProjectsPage: browseable gallery with filter by tech stack, type, status
- Featured on user's PublicProfilePage

5.11 Hackathon Module

Organizer Flow

28. Organizer creates hackathon: title, description, dates, mode, venue, team size limits, prizes, banner image
29. Hackathon published; appears on HackathonsPage with countdown to registration deadline
30. Teams can register for the hackathon from HackathonDetailPage (leader action)
31. Organizer dashboard shows registered teams, participant count, announcements

Student Flow

- Browse hackathons with filter: upcoming, ongoing, past; mode; tags
- Register an existing team or create a new team within hackathon context

- Hackathon-linked teams appear on HackathonDetailPage team list

5.12 Notifications Module

- Notification types: new_application, application_accepted, application_rejected, invitation_received, invitation_accepted, new_message, new_review, team_update, hackathon_reminder, system_announcement
- Real-time delivery via socket event 'notification:new'
- Notification bell shows unread count badge (updates in real-time)
- NotificationDropdown shows last 10; link to /notifications for full list
- Batch mark-as-read, individual mark-as-read, delete
- Email digest: daily summary for users with email_notifications enabled
- Auto-purge notifications older than 90 days via MongoDB TTL index

5.13 Ratings & Reviews

- A user can review a teammate only after sharing a completed team together
- Rating: 1–5 stars + optional written comment + predefined positive tags (Good Communicator, Reliable, Skilled, Team Player, Creative, Punctual)
- Average rating displayed on PublicProfilePage with review count
- A user cannot review themselves; one review per reviewer per team context
- Reviews can be reported; admin can hide flagged reviews

5.14 Reporting System

- Report button available on: user profiles, team pages, messages, reviews
- Report reasons: spam, harassment, fake profile, inappropriate content, other
- Admin sees report queue in AdminReportsPage: filter by status/type, resolve with action log
- Actions: warn user, remove content, temporary ban, permanent ban
- Reporter notified of resolution outcome

5.15 Admin Dashboard

- Summary widgets: total users, new registrations (7d), active teams, open reports, hackathons live
- User management: search, view profile, assign/change role, ban/unban, delete account
- Team management: view, force-disband, feature team on landing page
- Hackathon management: approve/reject, feature, delete
- Reports queue: prioritised moderation workflow
- Skill management: add/edit/delete skills in the master Skill collection
- Email broadcast: send system announcements to all users or segmented groups

5.16 Analytics Dashboard

Admin Analytics

- User registration trend (daily/weekly/monthly line chart)
- Active users (DAU/WAU/MAU bar chart)
- Team formation rate (teams created per week)
- Application conversion rate (applications → acceptances)
- Top skills demanded vs available
- College distribution (pie chart)
- Hackathon engagement (teams per hackathon bar chart)

Student Analytics (own profile)

- Profile views (weekly)
- Applications sent / accepted / rejected
- Invitations received / accepted
- Match score trend over time

6. Matching Engine Design

6.1 Algorithm Overview

The HackMatch Skill Matching Engine computes a pairwise compatibility score between User A (the logged-in student) and User B (a candidate teammate). Scores are computed on-demand when User A browses the DiscoverPage and are cached for up to 6 hours in the user document (matchScore field for overall score; full breakdowns stored transiently in Redis or re-computed). The engine outputs a 0–100 score with dimensional breakdown.

6.2 Scoring Dimensions

Dimension	Weight	How Computed
Skill Complementarity	35%	Jaccard complement: skills B has that A lacks, normalised by team open role requirements
Skill Overlap	10%	Shared skills (for domain alignment, avoids total mismatch)
College Match	15%	Same college = 15 pts; same city = 7 pts; else 0
Year Proximity	10%	Abs(A.year – B.year) ≤ 1 = 10pts; ≤ 2 = 5pts; else 0
Interests Overlap	10%	Intersection(A.interests, B.interests) / Union × 10
Experience	10%	B.hackathonsAttended normalised to 0–10 scale
Availability	10%	B.availability == 'available' = 10; 'busy' = 5; 'not_looking' = 0

6.3 Score Formula

matchScore = Σ (dimensionScore × weight)

Skill Complementarity (35 pts):
  requiredSkills = team.openRoles.flatMap(role => role.skills)
  missingFromA = requiredSkills.filter(s => !A.skills.includes(s))
  complementScore = B.skills.filter(s => missingFromA.includes(s)).length / missingFromA.length
  → dimensionScore = complementScore × 35

Final score clipped to [0, 100] and rounded to integer.
Scores below 20 are filtered from the default discovery view (configurable).

6.4 Recommendation Flow

32. User opens DiscoverPage → GET /api/match/recommendations?page=1&limit=20
33. Server fetches paginated candidate pool: users where isActive=true, isEmailVerified=true, availability != 'not_looking', _id != currentUser._id
34. For each candidate, matchingEngine.computeMatchScore(currentUser, candidate) is called

35. Candidates sorted by score DESC; pagination applied
36. Response includes candidate profile snippet + score + score breakdown
37. Frontend renders UserCards with animated score bar, skill chips, quick-action buttons
38. Filters (skill, college, year, availability) applied as MongoDB query pre-filters to reduce computation pool

7. Real-Time Chat Architecture

7.1 Infrastructure

Vercel Serverless Functions do not support persistent TCP connections required by Socket.io. Therefore the Socket.io server runs as a separate long-lived Node.js process hosted on Railway, Render, or a small DigitalOcean Droplet. The React frontend connects to this WebSocket server independently of the REST API. The WebSocket server has READ access to MongoDB Atlas to authenticate socket connections via JWT.

7.2 Socket.io Event Architecture

Event Name	Direction	Payload	Purpose
connection	Client → Server	{ token }	Authenticate socket, join user's personal room
join:conversation	Client → Server	{ conversationId }	Join a 1:1 or group chat room
message:send	Client → Server	{ conversationId, type, content, mediaURL }	Send a message
message:new	Server → Client	{ message object }	Broadcast new message to room
message:read	Client → Server	{ conversationId, messageId }	Mark message as read
message:readUpdate	Server → Client	{ messageId, readBy }	Update read receipts for all room members
typing:start	Client → Server	{ conversationId }	User started typing
typing:stop	Client → Server	{ conversationId }	User stopped typing
typing:update	Server → Client	{ userId, isTyping }	Broadcast typing state to room
presence:online	Server → Client	{ userId }	User came online
presence:offline	Server → Client	{ userId, lastSeen }	User went offline
notification:new	Server → Client	{ notification object }	Push new notification to user's personal room
disconnect	Client → Server	auto	Update lastSeen, broadcast presence:offline

7.3 Conversation ID Strategy

1:1 Chats: conversationId = sorted join of two user IDs → 'uid1_uid2' (sorted alphabetically ensures same ID regardless of who initiates)

Group Chats: conversationId = 'team_' + team._id.toString()

Message documents use conversationId as the primary lookup field. Index on conversationId + createdAt (desc) enables O(log n) paginated message retrieval.

7.4 Message Persistence

- All messages persisted to MongoDB Message collection regardless of recipient online status
- On socket connect, server checks unread messages and emits buffered notification counts
- Frontend fetches REST endpoint GET /api/messages/:conversationId for history on open
- Socket delivers real-time updates; REST provides reliable history and pagination

7.5 Online Presence

- On socket connect: add userId to in-memory Set (or Redis Set for multi-instance); emit presence:online to all contacts
- On disconnect: remove from Set, update User.lastSeen, emit presence:offline
- REST endpoint GET /api/users/presence?ids=[...] returns current online status for a list of user IDs

8. API Documentation

Base URL (production): https://api.hackmatch.app/api | All requests: Content-Type: application/json | Auth: Bearer <accessToken> in Authorization header (or httpOnly cookie). All responses follow { success, data, message, pagination? } envelope.

8.1 Authentication Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
POST	/auth/register	{ email, password, firstName, lastName, college, degree, branch, year }	{ user, message: 'OTP sent' }	No	Email valid, password ≥8 chars, all fields required
POST	/auth/verify-otp	{ email, otp }	{ user, accessToken }	No	OTP 6 digits, not expired
POST	/auth/login	{ email, password }	{ user, accessToken }	No	Credentials match, account active & verified
POST	/auth/logout	{}	{ message: 'Logged out' }	Yes	Clears cookies, removes refreshToken
POST	/auth/refresh	{} (cookie)	{ accessToken }	No	Valid refreshToken cookie
POST	/auth/forgot-password	{ email }	{ message: 'OTP sent' }	No	Email must exist
POST	/auth/verify-reset-otp	{ email, otp }	{ resetToken }	No	OTP valid
POST	/auth/reset-password	{ resetToken, newPassword }	{ message: 'Password updated' }	No	resetToken valid, password ≥8 chars

8.2 User Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/users/me	—	{ user }	Yes	—
PATCH	/users/me	{ bio, college, year, interests, githubURL, linkedinURL, portfolioURL, availability }	{ user }	Yes	Field-level validation

Method	Endpoint	Request Body	Response	Auth	Validation
POST	/users/me/avatar	FormData: { avatar: File }	{ avatarURL }	Yes	Image only, max 5MB
GET	/users/:id	—	{ user, reviews, projects }	Yes	Public profile
GET	/users/search	?q=&skills=&college=&year=&page=&limit=	{ users, pagination }	Yes	q min 2 chars
GET	/users/presence	?ids=id1,id2,…	{ presence: {id: bool} }	Yes	Max 50 IDs
DELETE	/users/me	{ password }	{ message }	Yes	Soft delete, requires password confirm

8.3 Team Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/teams	?type=&skills=&hackathon=&college=&recruiting=&sort=&page=&limit=	{ teams, pagination }	Yes	—
POST	/teams	{ name, description, projectType, maxSize, openRoles, skills, hackathon?, isPublic }	{ team }	Yes (Leader)	name required, maxSize 2–10
GET	/teams/:id	—	{ team, members, openRoles }	Yes	—
PATCH	/teams/:id	{ name?, description?, openRoles?, isRecruiting?, status? }	{ team }	Yes (Leader/Admin)	Leader owns team
DELETE	/teams/:id	—	{ message }	Yes (Leader/Admin)	Notifies members
POST	/teams/:id/members/:userId	{ role }	{ team }	Yes (Admin)	Admin add member directly
DELETE	/teams/:id/members/:userId	—	{ team }	Yes (Leader/Admin)	Cannot remove leader

8.4 Application Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/applications	?teamId=&status=&page=	{ applications }	Yes (Leader for team, Student for own)	—
POST	/applications	{ teamId, role, coverMessage? }	{ application }	Yes (Student)	Not already member, not duplicate
PATCH	/applications/:id/withdraw	—	{ application }	Yes (Applicant)	Status must be pending
PATCH	/applications/:id/accept	—	{ application, team }	Yes (Leader)	Checks team not full
PATCH	/applications/:id/reject	{ message? }	{ application }	Yes (Leader)	—

8.5 Invitation Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/invitations/received	?status=&page=	{ invitations }	Yes	—
GET	/invitations/sent	?status=&page=	{ invitations }	Yes (Leader)	—
POST	/invitations	{ teamId, inviteeId, role?, message? }	{ invitation }	Yes (Leader)	Not already member, no duplicate
PATCH	/invitations/:id/accept	—	{ invitation, team }	Yes (Invitee)	Not expired
PATCH	/invitations/:id/reject	—	{ invitation }	Yes (Invitee)	—
DELETE	/invitations/:id	—	{ message }	Yes (Leader)	Cancel pending invitation

8.6 Message Endpoints (REST – History)

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/messages/:conversationId	?before=<messageId>&limit=20	{ messages, hasMore }	Yes	Must be participant
DELETE	/messages/:id	—	{ message }	Yes (Sender/	Within 60min for

Method	Endpoint	Request Body	Response	Auth	Validation
				Admin)	sender

8.7 Notification Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/notifications	?isRead=&type=&page=	{ notifications, unreadCount }	Yes	—
PATCH	/notifications/:id/read	—	{ notification }	Yes	—
PATCH	/notifications/read-all	—	{ message }	Yes	—
DELETE	/notifications/:id	—	{ message }	Yes	—

8.8 Project Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/projects	?userId=&techStack=&status=&page=	{ projects, pagination }	Yes	—
POST	/projects	{ title, description, techStack, githubURL?, demoURL?, collaborators?, hackathon?, isPublic, status }	{ project }	Yes	title required
GET	/projects/:id	—	{ project }	Yes	—
PATCH	/projects/:id	{ ...fields }	{ project }	Yes (Owner/ Admin)	—
DELETE	/projects/:id	—	{ message }	Yes (Owner/ Admin)	—
POST	/projects/:id/thumbnail	FormData: { thumbnail: File }	{ thumbnailURL }	Yes (Owner)	Image only, max 5MB

8.9 Hackathon Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/hackathons	?mode=&tags=&up	{ hackathons, pagination }	No	—

Method	Endpoint	Request Body	Response	Auth	Validation
		coming=&page=			
POST	/hackathons	{ title, description, mode, startDate, endDate, registrationDeadline, venue?, teamSizeMin, teamSizeMax, prizes?, tags? }	{ hackathon }	Yes (Organizer/Admin)	Dates validated
GET	/hackathons/:id	—	{ hackathon, registeredTeams }	No	—
PATCH	/hackathons/:id	{ ...fields }	{ hackathon }	Yes (Organizer/Admin)	—
DELETE	/hackathons/:id	—	{ message }	Yes (Organizer/Admin)	—
POST	/hackathons/:id/register	{ teamId }	{ hackathon }	Yes (Leader)	Before deadline, team size valid
POST	/hackathons/:id/banner	FormData: { banner: File }	{ bannerURL }	Yes (Organizer/Admin)	Image only, max 10MB

8.10 Match Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/match/recommendations	?skills=&college=&year=&availability=&page=&limit=	{ candidates: [{ user, matchScore, breakdown }] }	Yes	—
GET	/match/team/:teamId	?page=	{ candidates }	Yes (Leader)	Candidates for team's open roles

8.11 Review Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/reviews/user/:userId	?page=	{ reviews, averageRating }	Yes	—
POST	/reviews	{ revieweeId, teamId, rating,	{ review }	Yes	Must share completed

Method	Endpoint	Request Body	Response	Auth	Validation
		comment?, tags? }			team, not self
PATCH	/reviews/:id	{ rating?, comment?, tags? }	{ review }	Yes (Reviewer)	Within 30 days of creation
DELETE	/reviews/:id	—	{ message }	Yes (Reviewer/Admin)	—

8.12 Report Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
POST	/reports	{ targetType, targetId, reason, description? }	{ report }	Yes	—
GET	/reports	?status=&type=&page=	{ reports }	Yes (Admin)	—
PATCH	/reports/:id/resolve	{ resolution, action }	{ report }	Yes (Admin)	—

8.13 Admin Endpoints

Method	Endpoint	Request Body	Response	Auth	Validation
GET	/admin/users	?role=&college=&isBanned=&q=&page=	{ users }	Yes (Admin)	—
PATCH	/admin/users/:id/ban	{ reason, duration? }	{ user }	Yes (Admin)	—
PATCH	/admin/users/:id/unban	—	{ user }	Yes (Admin)	—
PATCH	/admin/users/:id/role	{ role }	{ user }	Yes (Admin)	—
GET	/admin/analytics	?from=&to=	{ registrations, dau, teamFormation, … }	Yes (Admin)	—
GET	/admin/skills	—	{ skills }	Yes (Admin)	—
POST	/admin/skills	{ name, category, icon? }	{ skill }	Yes (Admin)	—
PATCH	/admin/skills/:id	{ name?, category? }	{ skill }	Yes (Admin)	—
DELET	/admin/skills/:id	—	{ message }	Yes	Cascade

Method	Endpoint	Request Body	Response	Auth	Validation
E				(Admin)	remove from users

9. Security Architecture

9.1 Authentication Security

- Passwords hashed with bcrypt, cost factor 12 (≈250ms on modern hardware; resistant to brute force)
- JWT accessTokens: 15-minute expiry, HS256 signed with 256-bit secret, stored in httpOnly SameSite=Strict cookie
- Refresh tokens: 7-day expiry, SHA-256 hashed before DB storage, rotated on every use
- OTPs: 6-digit numeric, bcrypt-hashed, 10-minute TTL, 5-attempt rate limit
- Account lockout: 5 consecutive failed logins → 15-minute lockout, incremental backoff

9.2 Transport Security

- All traffic over HTTPS; HSTS header enforced via Helmet
- Helmet.js sets: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer, Permissions-Policy, CSP header
- CORS: strict allowlist (Netlify domain + localhost for dev); credentials: true

9.3 Input Security

- express-validator on all request bodies and query params with strict schema validation
- mongo-sanitize middleware strips $ and . from all inputs (NoSQL injection prevention)
- xss-clean sanitises HTML from string inputs (XSS prevention)
- Helmet CSP header prevents inline script execution on the frontend
- Multer enforces MIME type allowlist for uploads (image/jpeg, image/png, image/webp; application/pdf for files) and max file sizes

9.4 Rate Limiting

Route Group	Limit	Window	Strategy
Auth routes (login, register, OTP)	10 requests	15 minutes	IP-based; block after threshold
General API routes	100 requests	15 minutes	IP + userId combined key
File upload routes	20 requests	1 hour	IP-based
Admin routes	200 requests	15 minutes	IP + admin userId
Socket.io message events	30 messages	1 minute	Per socket connection

9.5 Authorization

- Every protected route validates accessToken via verifyAccessToken middleware
- Role-based access: requireRole('admin'), requireRole('organizer', 'admin') middleware
- Resource ownership: requireOwner middleware checks req.user._id === resource.owner/_id
- Team Leader operations: additional check that req.user._id === team.leader

9.6 Data Privacy

- passwordHash, emailOTP, refreshTokens fields excluded from all API responses via Mongoose .select('-passwordHash -emailOTP -refreshTokens')
- Soft delete on user accounts: isActive=false; data retained for 30 days then purged by cron
- GDPR-style data export endpoint (future): GET /api/users/me/export returns all user data as JSON
- Cloudinary asset deletion triggered on avatar/thumbnail replacement or account delete

10. UI / UX Design System

10.1 Design Philosophy

HackMatch should feel like a premium funded startup product — not a college project. The UI draws inspiration from Linear, Vercel, Raycast, and Notion for clean information architecture, and from Framer for motion quality. It must be mobile-first but equally excellent on desktop.

10.2 Color System

Token	Hex	Usage
--color-primary	#1E3A8A	Brand color – nav, headings, CTAs
--color-accent	#3B82F6	Interactive elements, links, score bars
--color-success	#10B981	Accepted status, online indicators
--color-warning	#F59E0B	Pending status, deadlines near
--color-danger	#EF4444	Rejected, ban, error states
--color-surface	#F8FAFC	Page background
--color-card	#FFFFFF	Card background
--color-border	#E2E8F0	Dividers, card borders
--color-text-primary	#0F172A	Headings, labels
--color-text-secondary	#475569	Body copy, subtitles
--color-text-muted	#94A3B8	Placeholders, timestamps

10.3 Typography

Scale	Size	Weight	Use
Display	48px / 3rem	800	Hero headlines
H1	36px / 2.25rem	700	Page titles
H2	28px / 1.75rem	600	Section headers
H3	22px / 1.375rem	600	Card titles, sub-sections
Body Large	18px / 1.125rem	400	Lead paragraphs
Body	16px / 1rem	400	Default body copy
Small	14px / 0.875rem	400	Labels, captions, timestamps
Micro	12px / 0.75rem	500	Tags, badges, chips

Primary Font: Inter (Google Fonts) – system fallback: -apple-system, sans-serif. Monospace: JetBrains Mono for code snippets, tech stack tags.

10.4 Animation Guidelines

- Page transitions: Framer Motion AnimatePresence with slide-fade (opacity 0→1, y: 16→0, duration 0.25s ease-out)
- Card hover: subtle scale(1.02) + shadow elevation increase, duration 200ms
- Card tilt effect: CSS perspective + rotateX/rotateY tracking mouse position (max ±8 degrees) using onMouseMove handler
- Scroll reveal: Intersection Observer API → trigger Framer Motion variants (opacity 0→1, y: 32→0, stagger children 0.1s)
- Animated statistics: useInView hook triggers count-up from 0 to target value over 1.5s with easeOut
- Hero blob: CSS radial-gradient blobs animated with keyframe transforms (translateX, translateY, borderRadius) at 8s loop
- Skeleton loading: Tailwind animate-pulse on gray placeholder shapes matching card layout
- Toast notifications: react-hot-toast with custom styling matching design system
- Button press: scale(0.96) on active state, 100ms duration
- Modal entrance: scale(0.92)→1, opacity 0→1, backdrop blur-in, 200ms
- Typing indicator: three bouncing dots using Framer Motion staggered y animation

10.5 Component Library Usage

- DaisyUI: btn, card, badge, modal, drawer, navbar, dropdown, tooltip, progress, stat, table, tabs, steps (form wizard)
- Flowbite: timeline (project history), accordion (FAQ on landing), datepicker (hackathon dates), file-input
- React Icons: Devicons for tech stack logos, HeroIcons for UI icons, SimpleIcons for social links
- All DaisyUI components overridden with Tailwind utility classes for brand colour adherence; no theme config file

10.6 Layout System

- Mobile-first breakpoints: sm: 640px, md: 768px, lg: 1024px, xl: 1280px, 2xl: 1536px
- Page container: max-w-7xl mx-auto px-4 sm:px-6 lg:px-8
- Grid layouts: CSS Grid with Tailwind grid-cols-1 md:grid-cols-2 xl:grid-cols-3
- Sidebar layouts (Chat, Admin): flex with fixed-width sidebar hidden on mobile, toggled via drawer

10.7 Key Screen Descriptions

Landing Page

- Full-viewport hero: animated gradient background, floating skill-chip elements, headline + subheadline + dual CTA buttons
- Animated stats bar: '12,000+ Students', '3,500+ Teams Formed', '800+ Hackathons', counting animation on scroll-into-view
- Feature cards: 3-column grid, each with icon, title, body; scroll-reveal stagger
- 'How it Works' section: 3-step horizontal timeline with connecting line animation
- Testimonial carousel (future): auto-scrolling student quotes
- Footer: link columns, social icons, copyright

Dashboard Page

- Welcome header with first name, current date, availability toggle
- 'Your Teams' horizontal scroll row of TeamCards
- 'Recommended Teammates' grid (top 6 by match score) with ViewAll link to DiscoverPage
- 'Upcoming Hackathons' compact list with countdown timers
- 'Recent Notifications' last 3 with mark-read links
- Quick actions: Create Team, Browse Teams, Find Teammates, Add Project

11. Deployment Architecture

11.1 Frontend – Netlify

- Build command: vite build | Publish directory: dist
- Environment variables: VITE_API_BASE_URL, VITE_SOCKET_URL set in Netlify dashboard
- _redirects file: /* /index.html 200 – enables client-side SPA routing
- Netlify CDN automatically distributes built assets globally across 200+ edge nodes
- Cache-Control: public, max-age=31536000, immutable on hashed JS/CSS bundles; max-age=0 on index.html
- Netlify Forms not used; form submissions go directly to Express API
- Branch deploys: main → production; develop → staging with password protection

11.2 Backend – Vercel Serverless Functions

- vercel.json routes all /api/* requests to Express app via @vercel/node runtime
- Express app exported as default from index.js (module.exports = app)
- Maximum function execution: 30 seconds (Vercel Pro limit); long-running match computation uses background processing pattern
- Environment variables: MONGODB_URI, JWT_SECRET, JWT_REFRESH_SECRET, CLOUDINARY_*, SMTP_* set in Vercel dashboard
- Connection caching: module-level global._mongoClientPromise pattern (see Section 3.1)
- Cold start mitigation: Vercel Fluid Compute (preview) or keep-alive ping from Netlify scheduled function every 5 minutes
- File uploads: Multer memory storage → immediately stream to Cloudinary; no disk writes on Vercel

11.3 Socket.io Server – Railway

- Separate Node.js service: node socket.js on Railway (always-on $5/month starter)
- SOCKET_PORT, MONGODB_URI, JWT_SECRET set as Railway environment variables
- CORS: allow Netlify frontend origin and API domain
- Health check endpoint: GET /health returns 200
- Socket server connects to same MongoDB Atlas cluster for JWT verification and message persistence

11.4 MongoDB Atlas

- Cluster tier: M10 (3-node replica set, 2GB RAM, 10GB storage) for MVP; upgrade to M20 at 10k users

- Region: AWS ap-south-1 (Mumbai) for lowest latency from Indian colleges
- Atlas Search indexes: User (firstName, lastName, college), Team (name, description), Project (title, description)
- Atlas Scheduled Trigger: daily job to expire old OTPs, purge old notifications, update hackathon statuses
- Backup: continuous cloud backup, 7-day retention, point-in-time recovery
- VPC Peering: configure for Vercel production environment to restrict Atlas network access

11.5 Cloudinary

- Upload preset: hackmatch_avatars (max 5MB, image only, auto-crop to 400×400, WebP conversion, q_auto:best)
- Upload preset: hackmatch_banners (max 10MB, image only, resize to 1200×400, WebP, q_auto)
- Upload preset: hackmatch_thumbnails (max 5MB, resize to 800×450, WebP, q_auto)
- Signed uploads via backend: Multer → memory buffer → cloudinary.uploader.upload_stream
- CDN delivery: Cloudinary global CDN with format=auto, quality=auto URL params
- Asset deletion: cloudinary.uploader.destroy(publicId) called on replacement or account deletion

12. MVP Roadmap

Phase 1 – Foundation (Weeks 1–4)

Goal: Core authentication, profile management, and infrastructure.

- Backend: MongoDB Atlas setup, Express app scaffold, all Mongoose models, auth module (register/login/OTP/JWT/refresh), user CRUD, Cloudinary integration, Multer, email service
- Frontend: Vite + React + Tailwind + DaisyUI setup, AuthContext, SocketContext, App.jsx routing, LandingPage, RegisterPage (3-step), LoginPage, VerifyOTPPage, ForgotPasswordPage, ResetPasswordPage, ProfilePage, EditProfilePage, ProtectedRoute, Navbar, Footer
- DevOps: MongoDB Atlas cluster live, Vercel deployment pipeline, Netlify deployment pipeline, environment variables configured, domain connected
- Deliverable: Users can register, verify email, login, build full profile with skills and social links, upload avatar

Phase 2 – Core Collaboration (Weeks 5–9)

Goal: Team creation, discovery, application, and invitation systems.

- Backend: Team CRUD, TeamApplication lifecycle, TeamInvitation lifecycle, Skill collection seed, Notification system, Matching Engine v1
- Frontend: TeamsPage, TeamDetailPage, CreateTeamPage (4-step wizard), ManageTeamPage, ApplicationModal, InviteModal, DiscoverPage with match scores, NotificationDropdown, NotificationsPage, UserCard, TeamCard, FilterPanel, Pagination, PublicProfilePage
- Deliverable: Complete team formation workflow – create → post roles → discover candidates → invite or receive applications → manage members

Phase 3 – Engagement (Weeks 10–13)

Goal: Real-time chat, hackathons, projects, reviews, reporting.

- Backend: Socket.io server on Railway, Message persistence, Hackathon CRUD, Project CRUD, Review system, Report system
- Frontend: ChatPage (sidebar + window), typing indicator, read receipts, presence dots; HackathonsPage, HackathonDetailPage, CreateHackathonPage; ProjectsPage, ProjectDetailPage, CreateProjectPage; ReviewModal, ReportModal; OrganizerDashboardPage, ManageHackathonPage
- Deliverable: Real-time team communication, hackathon discovery + team registration, project portfolio, peer reviews, content moderation tools

Phase 4 – Admin, Analytics & Polish (Weeks 14–16)

Goal: Admin dashboard, analytics, performance optimisation, launch preparation.

- Backend: Admin controllers, analytics aggregation pipelines, performance tuning (indexes audit, query explain plans), rate limiting final config, security audit
- Frontend: AdminDashboardPage, AdminUsersPage, AdminTeamsPage, AdminReportsPage, AdminAnalyticsPage (recharts), AdminHackathonsPage, SettingsPage, ErrorBoundary, full animation pass (Framer Motion on all pages), StatsCounter, loading skeletons, mobile responsiveness QA
- Launch prep: initAdmin.js seed script, GDPR privacy policy page, Terms of Service page, 404 page, SEO meta tags, Open Graph tags, Netlify preview and production split, final security pen test checklist
- Deliverable: Fully production-ready platform ready for beta launch to 5 partner colleges

13. Future Roadmap

13.1 AI Team Matching (v2)

- Replace rule-based matching engine with vector embedding similarity using OpenAI text-embedding-3-small
- Encode user profiles as semantic vectors; cosine similarity for ranked recommendations
- Fine-tune with historical 'team accepted / project succeeded' outcome data

13.2 GitHub Skill Verification

- OAuth GitHub login → fetch public repositories via GitHub API
- Analyse repository languages, commit frequency, star count, contribution graph
- Auto-suggest verified skills to user profile; display 'GitHub Verified' badge
- Calculate experience score per language based on commit history

13.3 Resume / LinkedIn Analysis

- Upload resume PDF → extract text via pdf-parse → GPT-4o extracts skills, experience, education
- Pre-fill profile fields from extracted data with user confirmation step

13.4 AI Team Builder

- Natural language team requirement input: 'I need a team to build an ML-based fintech app for a 48-hour hackathon'
- GPT-4o generates ideal team composition; matching engine finds best candidates for each role
- One-click bulk invitations to recommended candidates

13.5 AI Mentor

- Chatbot trained on hackathon winning strategies, project structuring, team dynamics best practices
- Context-aware: knows user's current team, project, hackathon; gives personalised advice

13.6 AI Career Recommendations

- Based on skills, projects, hackathon history: recommend job roles, learning paths, open-source projects to contribute to
- Integration with recruiter module: recruiters pay to access anonymised talent pool matched to their requirements

13.7 Mobile App

- React Native app sharing business logic and API with web platform

- Push notifications via Expo Notifications + FCM
- Offline-first message drafts with sync on reconnect

13.8 Institutional Partnerships

- College admin dashboard: verify student email domains, publish official college hackathons, track student engagement
- Incubator / startup cell integration: seed funding listings, mentor profiles, startup team formation
- Placement cell integration: skill gap analysis, company requirements matching

End of Product Requirements Document

HackMatch – Version 1.0 – Confidential